

Assignment 1

Optimization Methods Case Studies

Prepared by: Ratan Kumar Mahato

Roll No: 2501AI18

Data Science and Artificial Intelligence (DSAI)

Indian Institute of Technology (IIT) Patna

1. The Big-M Simplex Method

Case Study Definition

A pharmaceutical company needs to produce a daily minimum of 1000 mg of Vitamin C and 1200 mg of Calcium. They can achieve this by mixing two supplements, x_1 and x_2 .

- Supplement x_1 costs 2 INR per unit and contains 20 mg of Vitamin C and 10 mg of Calcium.
- Supplement x_2 costs 3 INR per unit and contains 10 mg of Vitamin C and 30 mg of Calcium.

The objective is to find the optimal mix of these supplements to minimize the total cost while meeting the dietary constraints.

Mathematical Formulation

Objective Function:

$$\text{Minimize } Z = 2x_1 + 3x_2$$

Subject to Constraints:

$$20x_1 + 10x_2 \geq 1000 \text{ (Vitamin C)}$$

$$10x_1 + 30x_2 \geq 1200 \text{ (Calcium)}$$

$$x_1, x_2 \geq 0$$

Conversion to Standard Form

To apply the Big-M method, the "greater than or equal to" constraints must be converted into equalities. We introduce surplus variables (S_1, S_2) to represent the excess amount above the minimum, and artificial variables (A_1, A_2) to provide an initial basic feasible solution. A very large penalty, M , is assigned to the artificial variables in the objective function.

$$\text{Minimize } Z = 2x_1 + 3x_2 + 0S_1 + 0S_2 + MA_1 + MA_2$$

$$20x_1 + 10x_2 - S_1 + A_1 = 1000$$

$$10x_1 + 30x_2 - S_2 + A_2 = 1200$$

$$x_1, x_2, S_1, S_2, A_1, A_2 \geq 0$$

Solution Methodology

The problem is solved using the Simplex algorithm with the following iterative steps:

1. **Initialization:** A tableau is constructed with A_1 and A_2 as the initial basic variables.
2. **Indicator Row Calculation:** For each column, we calculate $Z_j - C_j$. Since this is a minimization problem, we select the column with the most positive value as the entering variable.
3. **Minimum Ratio Test:** We divide the Right-Hand Side (RHS) values by the positive coefficients of the entering column. The row with the minimum positive ratio determines the leaving variable.
4. **Pivoting:** Row operations are performed to make the pivot element 1 and all other elements in the entering column 0.
5. **Optimality:** The process repeats until all $Z_j - C_j \leq 0$, indicating the optimal solution has been found.

Final Optimization Results

After computational execution of the Big-M tableau operations, the algorithm converges to the following optimal solution:

- **Optimal x_1 :** 36.00 units
- **Optimal x_2 :** 28.00 units
- **Minimum Total Cost (Z):** 156.00 INR

First Problem — Code and Output

```
File Edit Selection View Go Run Terminal Help
assignment1.py
1 import numpy as np
2 M = 10000.0
3 c_j = np.array([2.0, 3.0, 0.0, 0.0, M, M])
4 tableau = np.array([
5     [20.0, 10.0, -1.0, 0.0, 1.0, 0.0, 1000.0],
6     [10.0, 30.0, 0.0, -1.0, 0.0, 1.0, 1200.0]
7 ])
8
9 basic_vars = [4, 5]
10
11 print("--- Starting Big-M Simplex Optimization ---")
12
13 iteration = 0
14 while True:
15     iteration += 1
16     print(f"Iteration {iteration}:")
17     cb = c_j[basic_vars]
18
19 Problems Output Debug Console Terminal Ports
20 PS C:\Users\ratan\OneDrive\Desktop\Kodr> python -u "c:\Users\ratan\OneDrive\Desktop\Kodr\assignment1.py"
--- Starting Big-M Simplex Optimization ---
Iteration 1:
Entering Variable Index: 1
Leaving Variable Index: 5 (Row 1)
Iteration 2:
Entering Variable Index: 0
Leaving Variable Index: 4 (Row 0)
Iteration 3:
Optimality reached! All Z_j - C_j <= 0.
--- Final Optimization Results ---
Optimal x1 (Supplement 1): 36.00
Optimal x2 (Supplement 2): 28.00
Minimum Total Cost (Z): 156.00 INR
PS C:\Users\ratan\OneDrive\Desktop\Kodr>
```

```
File Edit Selection View Go Run Terminal Help
assignment1.py
1 import numpy as np
2 M = 10000.0
3 c_j = np.array([2.0, 3.0, 0.0, 0.0, M, M])
4 tableau = np.array([
5     [20.0, 10.0, -1.0, 0.0, 1.0, 0.0, 1000.0],
6     [10.0, 30.0, 0.0, -1.0, 0.0, 1.0, 1200.0]
7 ])
8
9 basic_vars = [4, 5]
10
11 print("--- Starting Big-M Simplex Optimization ---")
12
13 iteration = 0
14 while True:
15     iteration += 1
16     print(f"Iteration {iteration}:")
17     cb = c_j[basic_vars]
18     z_j = np.dot(cb, tableau[:, :-1])
19     z_j_c_j = z_j - c_j
20
21     if np.all(z_j_c_j <= 1e-7):
22         print("Optimality reached! All Z_j - C_j <= 0.")
23         break
24     entering_idx = np.argmax(z_j_c_j)
25     print(f"Entering Variable Index: {entering_idx}")
26
27     ratios = []
28     for i in range(tableau.shape[0]):
29         if tableau[i, entering_idx] > 0:
30             ratios.append(tableau[i, -1] / tableau[i, entering_idx])
31         else:
32             ratios.append(np.inf)
33
34     if np.all(ratios == np.inf):
35         raise ValueError("The solution is unbounded.")
36     leaving_row = np.argmin(ratios)
37     leaving_var = basic_vars[leaving_row]
38     print(f"Leaving Variable Index: {leaving_var} (Row {leaving_row})")
39     basic_vars[leaving_row] = entering_idx
40
41     pivot_element = tableau[leaving_row, entering_idx]
42     tableau[leaving_row, :] = tableau[leaving_row, :] / pivot_element
43
44     for i in range(tableau.shape[0]):
45         if i != leaving_row:
46             multiplier = tableau[i, entering_idx]
47             tableau[i, :] = tableau[i, :] - (multiplier * tableau[leaving_row, :])
48
49     optimal_x = np.zeros(len(c_j))
50     for i in range(len(basic_vars)):
51         var_idx = basic_vars[i]
52         optimal_x[var_idx] = tableau[i, -1]
53
54     min_cost = np.dot(optimal_x, c_j)
55
56     print(f"--- Final Optimization Results ---")
57     print(f"Optimal x1 (Supplement 1): {optimal_x[0]:.2f}")
58     print(f"Optimal x2 (Supplement 2): {optimal_x[1]:.2f}")
59     print(f"Minimum Total Cost (Z): {min_cost:.2f} INR")
```

2. Transportation Problem (VAM & MODI)

Case Study Definition

A logistics division manages distribution from three central factories (F_1, F_2, F_3) to four regional warehouses (W_1, W_2, W_3, W_4). The goal is to determine the optimal shipping quantities to minimize total freight costs while strictly meeting supply limits and demand requirements.

Supply Capacities: $F_1 = 15, F_2 = 25, F_3 = 10$ (Total = 50)

Warehouse Demands: $W_1 = 5, W_2 = 15, W_3 = 15, W_4 = 15$ (Total = 50)

Transportation Cost Matrix (INR per unit):

Source \ Dest	W1	W2	W3	W4
F1	10	2	20	11
F2	12	7	9	20
F3	4	14	16	18

Solution Methodology: Step 1 - VAM

To find the Initial Basic Feasible Solution (IBFS), we employ Vogel's Approximation Method (VAM):

- Penalty Calculation:** For each row and column, we calculate the penalty (the difference between the lowest and second-lowest cost).
- Allocation:** We identify the row or column with the maximum penalty and allocate the maximum possible quantity to the cell with the lowest cost in that row/column.
- Adjustment:** The satisfied supply or demand is crossed out, and penalties are recalculated for the remaining matrix until all units are allocated.

Solution Methodology: Step 2 - MODI Method

To verify and improve the VAM solution, the Modified Distribution (MODI) or $u-v$ method is utilized:

- Dual Variables:** We calculate multipliers u_i for rows and v_j for columns using the occupied cells, such that $c_{ij} = u_i + v_j$ (setting $u_1 = 0$).

2. **Opportunity Costs:** For all unoccupied cells, we calculate the penalty/opportunity cost: $\Delta_{ij} = c_{ij} - (u_i + v_j)$.
3. **Optimality Check:** If all $\Delta_{ij} \geq 0$, the solution is optimal.
4. **Loop Shifting:** If any $\Delta_{ij} < 0$, we identify the most negative cell and trace a closed loop through other occupied cells. We shift the maximum possible allocation (θ) along this loop, alternating addition and subtraction, to generate a new, lower-cost allocation matrix. This process repeats until optimality is achieved.

Final Optimization Results

After finding the IBFS via VAM and subsequently resolving negative opportunity costs via MODI closed-loop matrix modifications, the algorithm yields the final optimal shipping plan:

Source \ Dest	W1	W2	W3	W4
F1	0	15	0	0
F2	0	0	15	10
F3	5	0	0	5

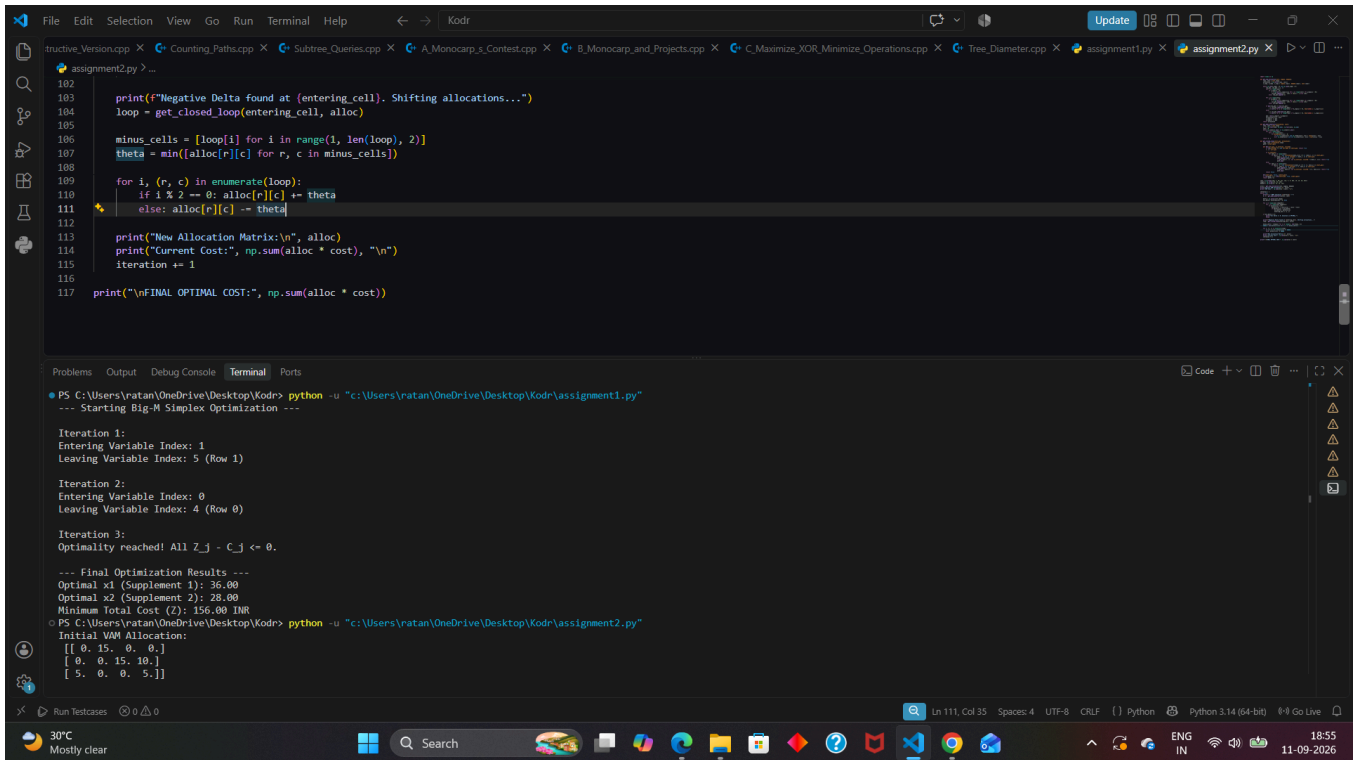
Final Optimal Total Transportation Cost: 435.00 INR

Second Problem · Code

```
File Edit Selection View Go Run Terminal Help ← → | Kodr Update
assignment2.py ~
47 def get_closed_loop(start_pos, allocation):
48     rows, cols = allocation.shape
49     path = [start_pos]
50
51     def dfs(curr_pos, is_vertical, visited):
52         if len(visited) > 3 and curr_pos == start_pos: return True
53         r, c = curr_pos
54
55         if is_vertical:
56             for next_r in range(rows):
57                 if next_r != r and (allocation[next_r][c] > 0 or (next_r, c) == start_pos):
58                     if (next_r, c) not in visited or (next_r, c) == start_pos:
59                         path.append((next_r, c))
60                         if dfs((next_r, c), not is_vertical, visited + [(next_r, c)]): return True
61                         path.pop()
62         else:
63             for next_c in range(cols):
64                 if next_c != c and (allocation[r][next_c] > 0 or (r, next_c) == start_pos):
65                     if (r, next_c) not in visited or (r, next_c) == start_pos:
66                         path.append((r, next_c))
67                         if dfs((r, next_c), not is_vertical, visited + [(r, next_c)]): return True
68                         path.pop()
69             return False
70
71     dfs(start_pos, False, [start_pos])
72     if len(path) == 1: return dfs(start_pos, True, [start_pos])
73     return path[:-1]
74
75 cost = np.array([[10, 2, 20, 11], [12, 7, 9, 20], [4, 14, 16, 10]])
76 supply = np.array([15, 25, 10])
77 demand = np.array([15, 15, 15])
78
79 alloc = get_van_allocation(cost, supply, demand)
80 print("Initial VAM Allocation:\n", alloc)
81 print("VAM Cost:", np.sum(alloc * cost), "\n")
82
83 iteration = 1
84 while True:
85     print(f"---- MODI Iteration (iteration) ----")
86     u, v = get_modi_modifiers(alloc, cost)
87
88     delta = np.zeros(cost.shape)
89     min_delta, entering_cell = 0, None
90
91     for i in range(cost.shape[0]):
92         for j in range(cost.shape[1]):
93             if alloc[i][j] == 0:
94                 delta[i][j] = cost[i][j] - (u[i] + v[j])
95                 if delta[i][j] < min_delta:
96                     min_delta = delta[i][j]
97                     entering_cell = (i, j)
98
99     if min_delta >= 0:
100         print("All Delta >= 0. Solution is OPTIMAL.")
101
102 Run Testcases 0 0 0
```

```
File Edit Selection View Go Run Terminal Help ← → | Kodr Update
assignment2.py ~
1 import numpy as np
2
3 def get_van_allocation(cost, supply, demand):
4     rows, cols = cost.shape
5     allocation = np.zeros((rows, cols))
6     s_copy, d_copy, c_copy = supply.copy(), demand.copy(), cost.copy()
7
8     while np.sum(s_copy) > 0 and np.sum(d_copy) > 0:
9         row_pen, col_pen = [], []
10         for i in range(rows):
11             if s_copy[i] > 0:
12                 v = np.sort((-c_copy[i][j] for j in range(cols) if d_copy[j] > 0))
13                 row_pen.append(v[1] - v[0] if len(v) > 1 else v[0])
14             else: row_pen.append(-1)
15
16         for j in range(cols):
17             if d_copy[j] > 0:
18                 v = np.sort((-c_copy[i][j] for i in range(rows) if s_copy[i] > 0))
19                 col_pen.append(v[1] - v[0] if len(v) > 1 else v[0])
20             else: col_pen.append(-1)
21
22         if max(row_pen) >= max(col_pen):
23             r = row_pen.index(max(row_pen))
24             c = min(j for j in range(cols) if d_copy[j] > 0, key=lambda j: c_copy[r][j])
25         else:
26             c = col_pen.index(max(col_pen))
27             r = min(i for i in range(rows) if s_copy[i] > 0, key=lambda i: c_copy[i][c])
28
29         qty = min(s_copy[r], d_copy[c])
30         allocation[r][c] = qty
31         s_copy[r] -= qty
32         d_copy[c] -= qty
33         return allocation
34
35 def get_modi_modifiers(allocation, cost):
36     rows, cols = cost.shape
37     u, v = np.full(rows, np.nan), np.full(cols, np.nan)
38     u[0] = 0
39     while np.isnan(u).any() or np.isnan(v).any():
40         for i in range(rows):
41             for j in range(cols):
42                 if allocation[i][j] > 0:
43                     if not np.isnan(u[i]) and np.isnan(v[j]): v[j] = cost[i][j] - u[i]
44                     elif np.isnan(u[i]) and not np.isnan(v[j]): u[i] = cost[i][j] - v[j]
45
46     return u, v
47
48 Run Testcases 0 0 0
```

Second Problem · Output



```
File Edit Selection View Go Run Terminal Help
tructive_Version.cpp X Counting_Paths.cpp X Subtree_Queries.cpp X A_Monocarp_s_Constest.cpp X B_Monocarp_and_Projects.cpp X C_Maximize_XOR_Minimize_Operations.cpp X Tree_Diameter.cpp X assignment1.py X assignment2.py X
assignment2.py > ...
102
103     print(f"Negative Delta found at {entering_cell}. Shifting allocations...")
104     loop = get_closed_loop(entering_cell, alloc)
105
106     minus_cells = [loop[i] for i in range(1, len(loop), 2)]
107     theta = min([alloc[r][c] for r, c in minus_cells])
108
109     for i, (r, c) in enumerate(loop):
110         if i % 2 == 0: alloc[r][c] += theta
111         else: alloc[r][c] -= theta
112
113     print("New Allocation Matrix:\n", alloc)
114     print("Current Cost:", np.sum(alloc * cost), "\n")
115     iteration += 1
116
117     print("\nFINAL OPTIMAL COST:", np.sum(alloc * cost))

Problems Output Debug Console Terminal Ports
PS C:\Users\ratan\OneDrive\Desktop\Kodr> python -u "c:\Users\ratan\OneDrive\Desktop\Kodr\assignment1.py"
--- Starting Big-M Simplex Optimization ---

Iteration 1:
Entering Variable Index: 1
Leaving Variable Index: 5 (Row 1)

Iteration 2:
Entering Variable Index: 0
Leaving Variable Index: 4 (Row 0)

Iteration 3:
Optimality reached! All  $Z_j - C_j \leq 0$ .

--- Final Optimization Results ---
Optimal x1 (Supplement 1): 36.00
Optimal x2 (Supplement 2): 28.00
Minimum Total Cost (Z): 156.00 INR
PS C:\Users\ratan\OneDrive\Desktop\Kodr> python -u "c:\Users\ratan\OneDrive\Desktop\Kodr\assignment2.py"
Initial WM Allocation:
[[ 0. 15.  0.  0.]
 [ 0.  0. 15. 10.]
 [ 5.  0.  0.  5.]]
```